rlmckenney / cst8918-w24-a06-terratest

Code Issues Pull requests 2 Actions Projects Security Insights

Watch 1

0 stars 128 forks 1 watching Branches Activity Tags

Public repository

2 Branches 0 Tags Go to file t Go to file Add file Code

Robert McKenney ADD output values to support testing 6a2f951 · 2 years ago

.gitignore	lab solution	2 years ago
.terraform.lock.hcl	lab solution	2 years ago
README.md	lab solution	2 years ago
README_solution.md	lab solution	2 years ago
init.sh	lab solution	2 years ago
main.tf	Split main.tf into smaller modules	2 years ago
main.tfplan	lab solution	2 years ago
outputs.tf	ADD output values to support testi...	2 years ago
providers.tf	Split main.tf into smaller modules	2 years ago
variables.tf	Split main.tf into smaller modules	2 years ago

README

CST8918 - DevOps: Infrastructure as Code
Prof: Robert McKenney

LAB-A05 Terraform Web Server

Background

This hands-on lab activity will explore using Terraform to deploy a simple web server on Azure.

Prerequisites

You will need the following tools installed locally on your laptop before you begin.

- git

- Azure CLI
- Terraform CLI
- an SSH key pair

Reference

- The main Terraform documentation
- The AzureRM Terraform Provider documentation
- The Azure public cloud documentation

Scenario

You are the sole DevOps engineer at a small professional services company. They have a small web application that runs a on a single server in the office. Anticipating traffic growth as the company expands they want to move the web server to the cloud. Azure has been selected as the public cloud provider.

Knowing that the complexity of the solution architecture will increase over time and the number of managed resources will grow as well, you want to "start on the right foot" by using Terraform to manage these infrastructure resources.As a first step you will create a simple web server on the latest supported version of Ubuntu linux. That server should be publicly accessible by both SSH and HTTP.

You will expand on this scenario later, but start with this basic setup.

Instructions

1. Create an architecture diagram

Review the scenario and create an architecture diagram for your proposed solution.

2. Create the project scaffolding

- create a new project folder called `cst8918-w24-A05-<your-username>`
- create a `.gitignore` file in that folder with the following content

```
# This is a minimal list, OK to add things per repo
# mac specific
.DS_Store
.ansible
.azure/
.bash_history
# don't check storage creds into GH
.boto
.cache
*.code-workspace
#  may contain gcloud files
.config
.gitconfig
.local
# .netrc contains secrets for service tokens
.netrc
*.plan
.sentinel
# .ssh dir may contain private keys
.ssh
.terrascan
# Terraform dot files
```

```
.terraform
.terraformrc
**/.terraform/*
.terraform.d
*.tfstate
*.tfstate.*
.vscode
.idea
.idea
```

- using the terminal, run `git init` in the project folder
- using the terminal, run `touch main.tf` to create the file that will contain your Terraform definitions
- make your initial git commit

```
git add .
git commit -m "initial commit"
```

As you complete each section below you should create a new git commit with a succinct but descriptive message.

3. Define required Terraform providers

The first block in your `main.tf` file should define the minimum version of Terraform plus any required provider modules. For this project you will need the `azurerm` and `cloudinit` providers.

```
# Configure the Terraform runtime requirements.
terraform {
  required_version = ">= 1.1.0"

  required_providers {
    # Azure Resource Manager provider and version
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0.2"
    }
    cloudinit = {
      source  = "hashicorp/cloudinit"
      version = "2.3.3"
    }
  }
}

# Define providers and their config params
provider "azurerm" {
  # Leave the features block empty to accept all defaults
  features {}
}

provider "cloudinit" {
  # Configuration options
}
```

Initialize Terraform

You should now run `terraform init` in the terminal. Terraform will validate your code and attempt to install the referenced provider modules. If any errors are reported, read them carefully to resolve the conflict -- usually a typo in a name.

4. Define resources

With the basic setup complete, you can define the specific resources from your architecture diagram.

4.1 Variables

It will be helpful to define a few variables that can be used consistently in the definition of other resources. These variables can have default values (or not) and can be overridden at runtime on the CLI. You will need:

- labelPrefix
- region
- admin_username

4.2 Resource Group

Define a resource group to logically contain all of the resources for this project. The name argument should be `"${var.labelPrefix}-A05-RG"`

4.3 Public IP Address

From the architecture diagram, the first peer resource in the resource group is the public IP address.

4.4 Virtual Network

Next you will need an Azure virtual network (VNet) with a CIDR range of `10.0.0.0/16`.

4.5 Subnet

Within the VNet, you will need to define a subnet with the CIDR range of `10.0.1.0/24`.

4.6 Security Group

The solution architecture design requires both SSH and HTTP access to the web server VM. The AzureRM provider allows for either defining security group rules as separate resources, or inlining them in the definition of the security group resource. Since we only have two rules to add, the inline approach will be preferable.

4.7 Virtual Network Interface Card (NIC)

The virtual machine will need a NIC that is connected to the public IP address.

4.8 Apply the security group

You could apply the security group rules to the whole subnet, or just to the web server. Since the rules that we defined are specific to the web server and may not be appropriate for other VMs that we put in the subnet, let's choose the second option.

4.9 Init Script

When the virtual machine is deployed, it will be a base Ubuntu server image. You will need to install the web server application on it. We will use apache for this. The **cloudinit** provider will let you define a shell script to run on the VM after the first boot.

Create a new file in your project folder called `init.sh` with these three lines of code.

```
#!/bin/bash
sudo apt-get update
```

```
sudo apt-get install -y apache2
```

Now define a **data** resource using the `cloudinit_config` resource type. You will use this resource when defining the VM in the next step.

4.10 Virtual Machine

You have all of the dependency resources defined and you can now define the web server virtual machine. Don't forget to include your SSH public key in the definition. Since this is a test deployment and not yet production, you should use a cheaper **B1s** sized VM.

Output Values

The resource definitions are now complete, but it would be nice to know how to validate the deployment. You will want to output a few essential values from the deployment:

- resource group name
- public IP address

5. Deploy and Verify

5.1 Deploy

> Use your college username when prompted for the *labelPrefix* variable.

```
terraform apply

var.labelPrefix
  Your college username. This will form the beginning of various resource names.

  Enter a value:
```

The Terraform CLI will parse your resource definitions and make sure that there are no syntax errors. If there are any, they will be displayed in red in your terminal. Correct them and try again.

Once everything is green, Terraform will display the planned changes. Type `yes` when prompted to deploy.

5.2 Verify

Once the deploy completes, you should see the two output values that you defined in your terminal.

- Use the resource group to visually inspect the resources in the Azure portal.
- Copy the public IP address into a browser to see the default Apache web page.
- Open a remote SSH session to the VM

```
ssh azureadmin@<public_ip_address>
```

Demo/Submit

Publish your project to GitHub

- create a new public remote repo on GitHub
- add the new GitHub repo as a remote on your local repo

- push your commits

Brightspace

Submit a link to your GitHub repo URL in the Brightspace assignment. The repo should include all of your Terraform project files and an image file with your architecture diagram from step 1, called `a05-architecture.png`

Clean up

Once you completed all of the tasks above, remember to clean-up any Azure resources that are no longer required.

```
terraform destroy
```

> Use your college username when prompted for the *labelPrefix* variable.

Releases

No releases published

Packages

No packages published

Contributors

No contributors

Languages

- **HCL** 98.8%
- **Shell** 1.2%